Prosit 3 - Boucle Requêtage

Rôles

Alexandre : Animateur

Youcef : Secrétaire

Antonin : Scribe

Maxime : Gestionnaire du temps

Contexte

La réunion avec les utilisateurs du projet CHU (médecins, soignants, chercheurs et administrateurs) a confirmé les analyses prévues. La modélisation conceptuelle est conservée et les traitements d'intégration ne nécessitent plus que les agrégations utiles. L'équipe engage la création du modèle physique, le remplissage des tables et l'écriture des requêtes analytiques dans Hive.

Les données seront actualisées tous les trois mois, tout comme les demandes de nouvelles analyses. Les utilisateurs ont fixé un temps de réponse maximal de deux minutes. Le projet dispose de trois jours pour produire les analyses ; les traitements concernent plusieurs millions de lignes et peuvent nécessiter plusieurs jointures.

Mots inconnus

Notion	Définition
HiveQL	Dialecte SQL de Hive utilisé pour interroger les données entreposées.
MapReduce	Modèle de calcul distribué fondé sur les étapes Map, Shuffle et Reduce.
Apache Tez	Moteur d'exécution de graphes de tâches pouvant exécuter les requêtes Hive.
Partitionnement	Répartition physique d'une table selon la valeur de colonnes, par exemple année.
Partition pruning	Exclusion des partitions inutiles lors de la lecture d'une requête.
Bucketing	Répartition des lignes en groupes selon un hachage de clé.
ORC	Format de stockage colonnaire compressé optimisé pour les lectures analytiques.
Vue matérialisée	Résultat pré-calculé et conservé pour accélérer certaines analyses.
EXPLAIN	Commande donnant le plan d'exécution d'une requête.
Benchmark	Mesure reproductible des performances dans des conditions fixées.
Cardinalité	Nombre de lignes ou de valeurs distinctes dans un ensemble.
Agrégation	Calcul synthétique : décompte, somme, moyenne ou proportion.

Problématique

Comment produire dans Hive les indicateurs d'activité, d'hospitalisation et les analyses régionales attendus par le CHU à partir du modèle dimensionnel alimenté, tout en garantissant la justesse des agrégations, la confidentialité des données de santé et un temps de réponse inférieur ou égal à deux minutes ?

Plan d'action

1. Comparaison des SGBDR et des bases NoSQL (1h).

2. Vérifier le modèle physique et le chargement des données (1h30) : Contrôler les tables, les grains, les correspondances et les volumes nécessaires au requêtage.

3. Construire les requêtes correspondant aux indicateurs métier (3h) : Traduire les analyses validées en requêtes et préciser les calculs et dénominateurs.

4. Étudier et appliquer les optimisations Hive (2h) : Examiner les plans d'exécution et comparer les stratégies d'accès aux données.

5. Définir le protocole de mesure du seuil de deux minutes (1h30) : Préparer un benchmark répétable et préciser comment interpréter les écarts.

Réalisation

1. Comparaison des SGBDR et des bases NoSQL

1.1 Différences entre SGBDR et NoSQL

Les systèmes de gestion de bases de données relationnelles (SGBDR) et les bases NoSQL répondent à des besoins différents en matière de stockage et de traitement des données.

Les SGBDR, comme PostgreSQL ou MySQL, organisent les données sous forme de tables reliées par des clés primaires et étrangères. Ils utilisent principalement SQL pour effectuer les requêtes et garantissent généralement des transactions respectant les propriétés ACID.

Les bases NoSQL proposent des modèles de stockage plus variés, souvent adaptés aux données volumineuses, semi-structurées ou distribuées sur plusieurs machines. Elles peuvent privilégier la disponibilité, la scalabilité horizontale ou la flexibilité du schéma selon les besoins.

On distingue quatre grandes familles de bases NoSQL :

Type de NoSQL	**Organisation**	**Exemple**	**Cas d'usage**
Clé-valeur	Association d'une clé unique à une valeur	Redis	Cache et accès rapides
Documents	Stockage de documents JSON ou BSON	MongoDB	Données médicales semi-structurées
Colonnes larges	Organisation en familles de colonnes	Cassandra, HBase	Données massives distribuées
Graphes	Stockage de nœuds et de relations	Neo4j	Analyse des relations entre entités

Dans notre projet CHU, l'objectif principal est de traiter de grands volumes de données médicales afin de produire des indicateurs statistiques. Nous utilisons donc Hive sur Hadoop pour exécuter des requêtes analytiques sur des données stockées dans HDFS.

Hive n'est pas une base NoSQL au même titre que MongoDB. Il s'agit d'un système d'entrepôt de données permettant d'interroger des données distribuées avec un langage proche de SQL : HiveQL.

Cette solution est adaptée aux besoins du projet grâce à son intégration avec Hadoop, au partitionnement des données et à l'utilisation de formats de stockage optimisés comme ORC.

1.2 Propriétés ACID et BASE

Les propriétés ACID et BASE correspondent à deux approches de la gestion de la cohérence et de la disponibilité des données.

ACID désigne quatre propriétés transactionnelles :

- **Atomicité :** une transaction est exécutée entièrement ou annulée.
- **Cohérence :** une transaction préserve les contraintes d'intégrité définies.
- **Isolation :** les transactions concurrentes ne doivent pas interférer de manière incorrecte.
- **Durabilité :** une fois validées, les modifications sont conservées même en cas de panne.

Ces propriétés sont particulièrement importantes dans les applications transactionnelles, notamment pour garantir la fiabilité des opérations médicales.

BASE correspond à une approche rencontrée dans certains systèmes distribués :

- **Basically Available :** le système privilégie sa disponibilité.
- **Soft State :** l'état du système peut évoluer temporairement sans nouvelle intervention.
- **Eventually Consistent :** les différentes copies des données convergent vers un état cohérent à terme, en l'absence de nouvelles modifications.

Contrairement à ACID, BASE accepte donc dans certaines architectures une cohérence différée afin de faciliter la disponibilité et la distribution.

Cependant, il ne faut pas considérer que tous les SGBDR sont exclusivement ACID et que toutes les bases NoSQL sont BASE. Les garanties dépendent du système choisi et de sa configuration.

Dans notre projet, la fiabilité des données médicales reste essentielle. Les traitements ETL doivent garantir la cohérence des informations avant leur intégration dans les tables analytiques Gold.

1.3. Théorème CAP

Le théorème CAP décrit les contraintes rencontrées par les systèmes de données distribués.

Il repose sur trois propriétés :

- **Consistency (C) :** les lectures obtiennent les dernières données validées selon un modèle de cohérence forte.
- **Availability (A) :** chaque requête adressée à un nœud opérationnel reçoit une réponse.
- **Partition Tolerance (P) :** le système continue à fonctionner malgré une interruption de communication entre certains nœuds.

Le théorème CAP indique qu'en présence d'une partition réseau, un système distribué ne peut pas garantir simultanément une cohérence forte et une disponibilité totale.

Certains systèmes privilégient la cohérence lors d'une partition, tandis que d'autres favorisent la disponibilité en acceptant temporairement des données potentiellement anciennes.

Dans notre architecture Hadoop, la répartition des données permet de traiter de grands volumes. Toutefois, le théorème CAP ne permet pas à lui seul de déterminer les performances de Hive : celles-ci dépendent également du moteur d'exécution, du stockage et des optimisations physiques.

1.4. Choix technologique pour le projet CHU

Le projet CHU nécessite principalement des analyses statistiques sur plusieurs millions de lignes, avec des mises à jour trimestrielles et un temps de réponse maximal de deux minutes.

L'utilisation de Hive sur Hadoop est pertinente pour plusieurs raisons :

- Le stockage des données dans HDFS permet de gérer des volumes importants.
- HiveQL facilite la réalisation des agrégations et des jointures analytiques.
- Le partitionnement permet de réduire les volumes lus lors des requêtes.
- Le format ORC optimise la lecture des colonnes nécessaires aux analyses.
- Les vues matérialisées permettent de précalculer certains indicateurs fréquemment utilisés.

2. Vérifier le modèle physique et le chargement des données

Le Prosit 2 a fixé un grain distinct par événement : une ligne par consultation, une par séjour hospitalier, une par note de satisfaction et une par décès. Cette séparation évite de multiplier les événements lors des jointures. Nous vérifions d'abord les dimensions communes et les partitions Gold, sans modifier la modélisation validée avec les utilisateurs.

Table Gold	Clés / axes attendus	Contrôle
fait_consultation	id_temps, etablissement_sk, diagnostic_sk, professionnel_sk	Un identifiant de consultation ne doit pas être compté deux fois.
fait_hospitalisation	id_temps, etablissement_sk, patient_sk, diagnostic principal	Un séjour = un fait ; attention aux diagnostics multiples.
dim_patient	patient_sk, sexe, âge ou date de naissance protégée	Tranches d'âge calculées à la date du séjour.
dim_temps	id_temps, date_complete, mois, annee	Correspondance unique pour chaque date.
dim_etablissement / dim_region	etablissement_sk, code_region et validité	Jointure temporelle non multiplicative.
fait_deces / fait_satisfaction	date, région et mesure selon la granularité	Aucune jointure nominative décès–patient.

Pour les séjours comportant plusieurs diagnostics, la table de liaison séjour–diagnostic du Prosit 2 ne doit pas être jointe directement à un décompte global sans déduplication. Pour une analyse par diagnostic, on peut compter les identifiants de séjours distincts, sous réserve de l'identifiant disponible.

HiveQL : vérifications initiales

```
SHOW TABLES IN gold;
DESCRIBE FORMATTED gold.fait_consultation;
SHOW PARTITIONS gold.fait_consultation;
SELECT annee, COUNT(*) AS nb_lignes
FROM gold.fait_consultation
GROUP BY annee ORDER BY annee;
```

Le format ORC convient aux colonnes nécessaires à l'analyse, et le partitionnement annuel n'a d'intérêt que si la clause de filtrage exploite la colonne année réellement déclarée comme partition. La présence d'une partition ne garantit pas sa lecture sélective : nous la vérifierons dans EXPLAIN.

3. Construire les requêtes correspondant aux indicateurs métier

Les six indicateurs obligatoires sont traités ci-dessous, suivis de deux analyses optionnelles. Le mot « taux » du cahier des charges n'identifie pas systématiquement un dénominateur : les requêtes fournissent d'abord un nombre ou une proportion précisément nommée. Un véritable taux rapporté aux patients éligibles exige une population de référence validée par les métiers.

Besoin demandé	Sortie calculée	Point de vigilance
Consultations d'un établissement X sur Y	Volume par établissement et période	Taux seulement si dénominateur disponible.
Consultations pour diagnostic X sur Y	Volume et part des consultations observées	Le code diagnostic doit être normalisé.
Hospitalisations globales sur Y	Nombre de séjours	Distinguer séjours et patients uniques.
Hospitalisations par diagnostic	Nombre de séjours distincts / diagnostic	Éviter les doublons de la liaison séjour–diagnostic.
Hospitalisations par sexe et âge	Nombre par sexe et tranche d'âge	Âge au moment du séjour ; effectifs protégés.
Consultations par professionnel	Nombre d'actes / praticien	Ne pas exposer d'identité non autorisée.
Décès 2019 par région (optionnel)	Nombre par région	Provenance INSEE, données agrégées.
Satisfaction 2020 par région (optionnel)	Moyenne pondérée ou part positive	Échelle des notes et seuil à confirmer.

Consultations pour un établissement et une année. Nous filtrons directement le fait sur la colonne partitionnée afin de réduire les données lues. La jointure vers le calendrier permet ensuite un regroupement mensuel.

HiveQL : consultations / établissement / période

```
SELECT f.etablissement_sk, t.mois,
       SUM(f.nb_consultation) AS nb_consultations
FROM gold.fait_consultation f
JOIN gold.dim_temps t ON f.id_temps = t.id_temps
WHERE f.annee = 2020 AND f.etablissement_sk = 125
GROUP BY f.etablissement_sk, t.mois
ORDER BY t.mois;
```

Consultations par diagnostic. La seconde mesure est une part parmi les consultations constatées durant la même année, et ne doit pas être étiquetée « taux de consultation des patients » sans validation métier.

HiveQL : diagnostic et part relative

```
WITH total AS (
 SELECT SUM(nb_consultation) AS n
 FROM gold.fait_consultation WHERE annee = 2020
)
SELECT d.code_diagnostic, SUM(f.nb_consultation) AS n,
       ROUND(100.0 * SUM(f.nb_consultation) /
             NULLIF(MAX(total.n), 0), 2) AS part_pct
FROM gold.fait_consultation f
```

```
JOIN gold.dim_diagnostic d ON f.diagnostic_sk = d.diagnostic_sk
CROSS JOIN total
WHERE f.annee = 2020 AND d.code_diagnostic = 'X'
GROUP BY d.code_diagnostic;
```

Hospitalisations globales et par diagnostic. Pour le décompte global, les faits de séjour suffisent. L'analyse détaillée utilise le diagnostic principal si le modèle le fournit ; sinon une table de correspondance doit être utilisée avec un comptage distinct des séjours.

HiveQL : hospitalisations et diagnostic principal

```
SELECT annee, COUNT(*) AS nb_sejours
FROM gold.fait_hospitalisation
WHERE annee BETWEEN 2019 AND 2020
GROUP BY annee ORDER BY annee;

SELECT d.code_diagnostic, COUNT(*) AS nb_sejours
FROM gold.fait_hospitalisation h
JOIN gold.dim_diagnostic d ON h.diagnostic_sk = d.diagnostic_sk
WHERE h.annee = 2020
GROUP BY d.code_diagnostic
ORDER BY nb_sejours DESC;
```

Hospitalisations par sexe et âge. L'âge doit correspondre au moment du séjour. Dans cette maquette, on suppose qu'une tranche d'âge à la date de l'événement a été préparée en Silver/Gold, comme préconisé au Prosit 2. Les groupes très petits seront masqués lors de la restitution.

HiveQL : sexe et tranche d'âge (champ à confirmer)

```
SELECT p.sexe, h.tranche_age_sejour,
       COUNT(*) AS nb_sejours
FROM gold.fait_hospitalisation h
JOIN gold.dim_patient p ON h.patient_sk = p.patient_sk
WHERE h.annee = 2020
GROUP BY p.sexe, h.tranche_age_sejour
ORDER BY p.sexe, h.tranche_age_sejour;
```

Consultations par professionnel. L'identifiant utilisé est technique ; l'autorisation de visualiser un professionnel nominativement relève des droits définis avec le responsable métier.

HiveQL : activité par professionnel

```
SELECT professionnel_sk,
       SUM(nb_consultation) AS nb_consultations
FROM gold.fait_consultation
WHERE annee = 2020
GROUP BY professionnel_sk
ORDER BY nb_consultations DESC;
```

Les deux indicateurs optionnels exploitent leurs tables de faits indépendantes. La satisfaction doit tenir compte des nombres de réponses lorsqu'un fichier contient déjà des notes moyennes agrégées ; faire la moyenne simple de moyennes pourrait produire un résultat faux.

HiveQL : besoins optionnels, selon grain vérifié

```
-- Décès INSEE : uniquement si la table a un grain décès
SELECT r.nom_region, COUNT(*) AS nb_deces
FROM gold.fait_deces d
JOIN gold.dim_region r ON d.region_sk = r.region_sk
WHERE d.annee = 2019
GROUP BY r.nom_region;

-- Satisfaction : si nb_reponses et somme_notes existent
```

```sql
SELECT r.nom_region,
 ROUND(SUM(s.somme_notes) /
       NULLIF(SUM(s.nb_reponses), 0), 2) AS note_moyenne
FROM gold.fait_satisfaction s
JOIN gold.dim_region r ON s.region_sk = r.region_sk
WHERE s.annee = 2020
GROUP BY r.nom_region;
```

Si le fichier INSEE est chargé en agrégats, COUNT(*) doit être remplacé par SUM(nb_deces). Si la satisfaction contient une note par réponse, AVG(note) remplace la somme pondérée. Ces alternatives ne doivent pas être confondues dans une même mesure.

4. Étudier et appliquer les optimisations Hive

L'objectif des deux minutes exige de limiter les données lues et les déplacements inutiles. Nous procédons par ordre : projection de colonnes utiles, filtrage précoce, partitionnement, format ORC, statistiques du moteur, puis bucketing et pré-agrégations quand les mesures le justifient. Une optimisation n'est retenue que si elle améliore les mesures sur les mêmes données.

Levier	Application au CHU	Limite / vérification
Partitionnement année	WHERE f.annee = 2020 sur table Gold	EXPLAIN doit limiter les partitions lues.
ORC	Lecture des colonnes nécessaires seulement	Mesurer volume lu et durée.
Statistiques	ANALYZE TABLE après chargement	Vérifier compatibilité et plan.
Bucketing établissement	Répartition stable selon la clé établissement	Bénéfice dépend du moteur et de la taille.
Vue matérialisée	Agrégat consultations établissement/mois	Rafraîchissement après chargement trimestriel.
Jointures	Préférer dimensions petites, écarter colonnes inutiles	Éviter les jointures sur des faits à grains différents.

HiveQL : lecture du plan et statistiques (à adapter)

```sql
EXPLAIN
SELECT etablissement_sk, SUM(nb_consultation) AS n
FROM gold.fait_consultation
WHERE annee = 2020
GROUP BY etablissement_sk;

ANALYZE TABLE gold.fait_consultation
PARTITION (annee) COMPUTE STATISTICS;
```

Le Prosit 1 propose une table ORC transactionnelle, partitionnée par année et bucketée par établissement. Nous conservons cette orientation, tout en signalant que le nombre de buckets (8 dans la maquette) et les bénéfices attendus doivent être validés sur la distribution des données et les capacités du moteur Tez ; un bucket ne garantit pas mécaniquement une jointure plus rapide.

HiveQL : agrégat matérialisé (prérequis à vérifier)

```sql
CREATE MATERIALIZED VIEW gold.mv_consult_etab_mois AS
SELECT f.etablissement_sk, f.annee, t.mois,
       SUM(f.nb_consultation) AS nb_consultations
FROM gold.fait_consultation f
JOIN gold.dim_temps t ON f.id_temps = t.id_temps
GROUP BY f.etablissement_sk, f.annee, t.mois;
```

```
-- Après chaque alimentation Gold validée
ALTER MATERIALIZED VIEW gold.mv_consult_etab_mois REBUILD;
```

5. Définir le protocole de mesure du seuil de deux minutes

Nous proposons un benchmark avant/après sur la même version de données Gold, avec au minimum les six analyses obligatoires et, si chargées, les deux optionnelles. Les mesures doivent distinguer le temps d'exécution serveur et le temps perçu par l'utilisateur de Superset, qui inclut connexion et restitution.

Scénario	Comparaison	Critère
A : requête de référence	Sans pré-agrégat, partitions non exploitées si possible	Temps, lignes et octets lus.
B : filtrage / partition	Même SQL métier avec filtre partitionné	EXPLAIN et durée.
C : ORC / statistiques	Même jeu, plan et statistiques à jour	Durée et charge CPU/RAM.
D : vue matérialisée	Requête sur agrégat frais	Durée et exactitude identique.
E : répétitions	3 à 5 passages, signaler cache et démarrage	Médiane et maximum.

Commandes : supervision et protocole

```
# Contrôler la consommation pendant la mesure
docker stats
# Lire les événements de HiveServer2 et Tez
docker compose logs --tail=150 hiveserver2
# Dans Hive / Beeline, lancer la même requête N fois
EXPLAIN SELECT ...;  -- remplacer par la requête du test
```

Le critère est fixé à 120 secondes par analyse, pour un jeu de données et des ressources documentées. Nous mesurons également les éventuels échecs, la saturation mémoire, le nombre de fichiers lus et les opérations de jointure coûteuses.

Indicateur	Avant (s)	Après (s)	Seuil 120 s	État
Consultations / établissement	À mesurer	À mesurer	≤ 120	À évaluer lors de l'exécution
Consultations / diagnostic	À mesurer	À mesurer	≤ 120	À évaluer lors de l'exécution
Hospitalisations globales	À mesurer	À mesurer	≤ 120	À évaluer lors de l'exécution
Hospitalisations / diagnostic	À mesurer	À mesurer	≤ 120	À évaluer lors de l'exécution
Hospitalisations sexe / âge	À mesurer	À mesurer	≤ 120	À évaluer lors de l'exécution
Consultations / professionnel	À mesurer	À mesurer	≤ 120	À évaluer lors de l'exécution

Ce prosit définit les requêtes et le protocole sur un modèle de maquette ; les noms de colonnes restent à adapter au modèle physique. Les temps seront relevés lors de l'exécution sur les données chargées : à ce stade, le respect du seuil n'est donc pas démontré.

Le suivi comprend un tableau de bord technique simple (durées par requête, nombre de lignes produites, erreurs et date de rafraîchissement). Si le seuil est dépassé, nous examinons d'abord le plan EXPLAIN et les ressources Docker avant de multiplier les vues matérialisées.

Conclusion

Ce Prosit 3 nous a permis d'approfondir l'exploitation des données dans un environnement Big Data à travers Hive et Hadoop. Nous avons étudié les différences entre les SGBDR et les bases NoSQL, puis défini les requêtes nécessaires pour répondre aux besoins d'analyse du groupe CHU. L'utilisation du partitionnement, du format ORC, du bucketing et des vues matérialisées permet d'envisager une optimisation des performances afin de respecter le seuil de deux minutes fixé par les utilisateurs. Cela permet de constituer une base essentielle pour le Prosit 4, consacré à la visualisation des indicateurs et à leur présentation sous forme de tableaux de bord interactifs.